

EV CHARGING NETWORK APP (ECNA)

Graduation Project COMP4902
Computer Engineering
Konya Food And Agriculture University

Submitted by

Keramettin Kutnu 222010020155
Emre Erkut 222010020156
Yunus Emre Kahraman 222010020157

Date Submitted

17.05.2026

ABSTRACT

The rapid growth of the electric vehicle (EV) market has introduced critical challenges for drivers, particularly range anxiety and the fragmentation of charging infrastructure data. The EV Charging Network App (ECNA) is designed to resolve these issues by providing a unified, centralized platform that consolidates information on over 15,000 charging stations across Turkey. This project delivers a high-performance, mobile-responsive web application featuring an interactive map, real-time smart route optimization, and advanced filtering capabilities based on energy source, access type, and distance. Furthermore, the system is fully localized, supporting dynamic theme and language switching (Turkish/English) to enhance usability for both local and international users. By seamlessly integrating Google Maps Platform with a robust React/Vite architecture, ECNA not only accelerates the decision-making process for EV drivers but also establishes a scalable foundation for future IoT device integration and secure payment processing.

Table of Contents

1. INTRODUCTION	1
2. ANALYSIS AND DESIGN	1
2.1 UPDATED FUNCTIONAL REQUIREMENTS.....	1
2.2 UPDATED NON-FUNCTIONAL REQUIREMENTS.....	1
2.3 UPDATED USE CASE REQUIREMENTS.....	2
2.4 DESIGN CLASSES.....	3
2.5 SEQUENCE DIAGRAM	4
2.6 SOFTWARE ARCHITECTURE.....	5
3. PROTOTYPE DESCRIPTION.....	5
3.1 IMPLEMENTATION PLATFORM.....	5
3.2 MAPPING BETWEEN REQUIREMENTS AND IMPLEMENTED FUNCTIONS	5
3.3 IMPLEMENTATION DETAILS.....	6
3.4 ACTUAL DATABASE SCHEMA.....	6
4. TESTING.....	7
4.1 EXPECTED TEST SCENARIOS	7
4.2 UNIT TEST	7
4.3 FUNCTIONAL TEST	7
4.4 USABILITY TEST.....	7
5. DEPLOYMENT OF THE SYSTEM	8
6. LIMITATION OF THE SYSTEM	8
7. CONCLUSION AND FUTURE WORK.....	8
8. REFERENCE.....	8

1. Introduction

The transition to sustainable energy has accelerated the adoption of Electric Vehicles (EVs) globally and in Turkey. However, EV drivers face significant challenges, such as "range anxiety," difficulty in finding available and compatible charging stations, and inefficient routing. The EV Charging Network App (ECNA) was developed to address these issues by providing a comprehensive, high-performance platform that centralizes data for over 15,000 charging stations across Turkey.

This project is important because it bridges the information gap between various charging network operators and EV drivers, offering a unified, map-first experience. In this project, we successfully developed a robust, responsive web application that features an interactive Google Maps integration, smart route planning with station discovery along the route, advanced filtering (by energy source, access type, and distance), and full multi-language (Turkish/English) and theme (Dark/Light) support. We achieved high-performance rendering for thousands of data points and laid the foundation for future real-time IoT and payment integrations.

2. Analysis and Design

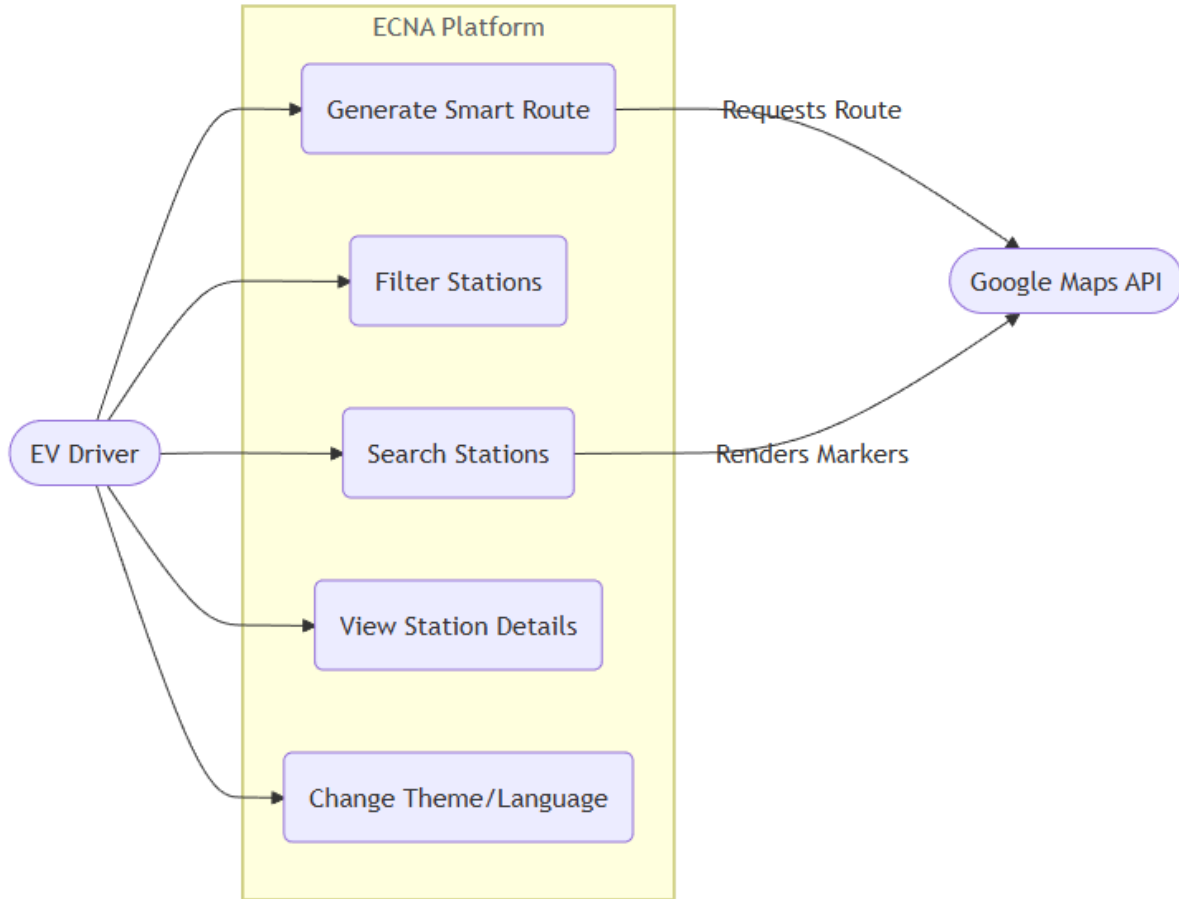
2.1 Updated Functional Requirements

- **Station Discovery & Map Interface:** The system shall display EV charging stations on an interactive Google Map, allowing users to view station details (power, sockets, pricing, green energy status).
- **Smart Route Planning:** The system shall calculate a driving route between an origin and a destination and filter/display charging stations located within a user-defined maximum deviation distance from the route.
- **Advanced Filtering:** Users shall be able to filter stations by energy source (e.g., Green Energy), service type (Public, Private), and distance from their current location.
- **Multi-Language Support (i18n):** The system shall support dynamic switching between Turkish and English without page reloads.
- **Theme Customization:** The system shall support Light and Dark modes with a cohesive UI design.
- **Real-time Location:** The system shall request and utilize the user's current GPS location for distance calculations and map centering.

2.2 Updated Non-Functional Requirements

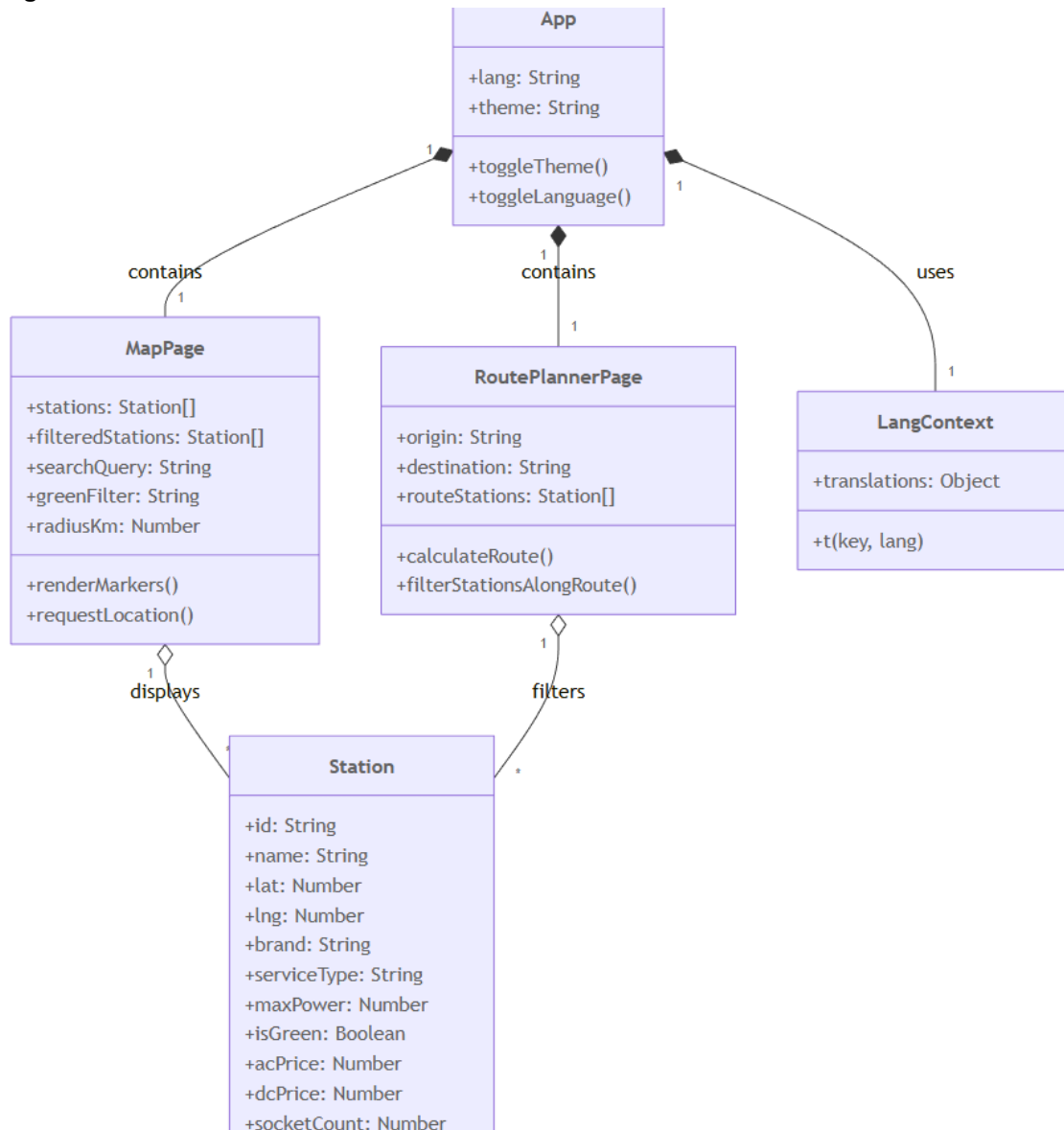
- **Performance:** The map must render efficiently, capable of handling a large dataset (~15,000 stations) without significant frame drops. We achieved this through bounding-box filtering and optimized state management.
- **Usability:** The UI must be responsive, working seamlessly on both desktop and mobile web browsers with a premium, map-centric design.
- **Scalability:** The frontend architecture (React + Vite + TypeScript) must be component-based to allow easy integration with future backend microservices (Node.js/Express) and IoT MQTT brokers.
- **Maintainability:** The codebase shall be strictly typed using TypeScript and centrally manage strings through an i18n dictionary.

2.3 Updated Use Case Requirements



1. **EV Driver:** Searches for nearby stations, applies filters (e.g., Green Energy only), views station details, switches language/theme, and creates a smart route.
2. **System/Map Layer:** Renders markers, calculates distance using Haversine formula, fetches routes via Google Directions API.

2.4 Design Classes

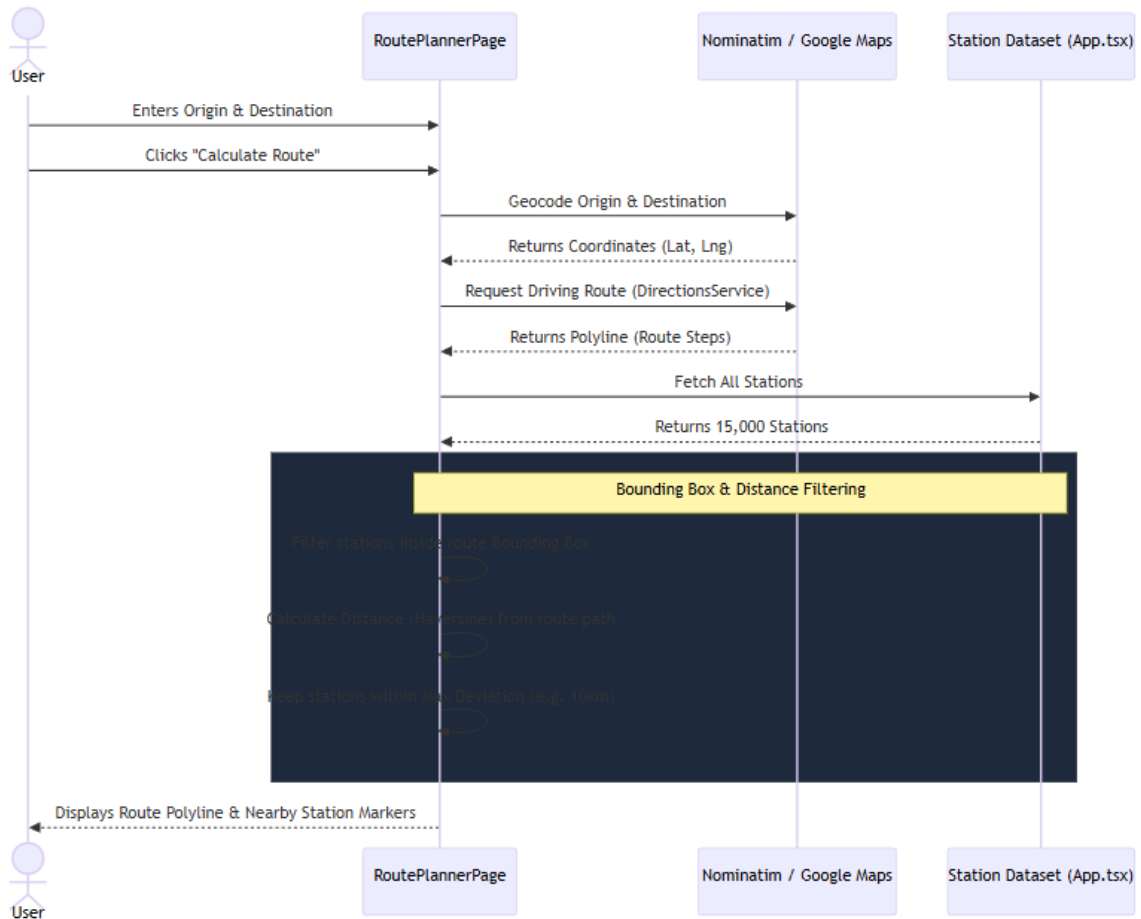


- **App**: Main controller handling Routing and global state (Theme, Language).
- **MapPage**: Handles Google Maps rendering, marker generation, user location, and filter states.
- **RoutePlannerPage**: Manages origin/destination inputs, Google Directions Service, and bounding-box station filtering.
- **LangContext**: Manages global localization state.
- **Station**: Data model containing `id`, `name`, `lat`, `lng`, `brand`, `serviceType`, `maxPower`, `isGreen`, `acPrice`, `dcPrice`, `socketCount`

2.5 Sequence Diagram

2.5.1 Sequence Diagram 1

Smart Route Generation Sequence:



1. User enters Origin and Destination and clicks "Calculate Route".
2. RoutePlannerPage requests geocodes for locations via OpenStreetMap Nominatim API.
3. RoutePlannerPage calls Google DirectionsService.route().
4. API returns polyline steps (coordinates).
5. System iterates over the route coordinates, filtering the local Station array to find stations within the maximum deviation radius.
6. System renders the route polyline and nearby station markers on the map.

2.6 Software Architecture

The current implementation utilizes a **Client-Side Single Page Application (SPA)** architecture, acting as the Presentation Layer.

- **Frontend:** React.js, TypeScript, Vite.
- **Styling:** Vanilla CSS with CSS Variables for theme management.
- **External Services:** Google Maps Platform (Maps JavaScript API, Directions API, Places/Geocoding API).
- **Data Layer:** Currently utilizes a normalized JSON dataset (stations.geocoded.json) built via custom Node.js scripts, acting as a mock database before transitioning to Firebase Firestore.

3. Prototype Description

The developed prototype is a responsive, web-based Single Page Application (SPA) designed to act as a centralized platform for Electric Vehicle (EV) drivers in Turkey. Built using React, TypeScript, and Vite, the prototype successfully integrates with the Google Maps Platform to render over 15,000 charging station data points in real-time. The core functionality centers around an interactive map interface that allows users to discover nearby stations, apply dynamic filters (such as green energy, public/private access, and maximum distance), and view detailed hardware and pricing specifications. A standout feature of the prototype is the "Smart Route" planner, which algorithmically filters and displays charging stations strictly within a user-defined deviation radius along a calculated driving path. Furthermore, the prototype implements a robust multi-language (Turkish and English) and theme-switching (Light/Dark mode) system, ensuring a high-quality, accessible, and user-centric experience across both desktop and mobile devices.

3.1 Implementation Platform

- **Software:** Visual Studio Code, Node.js (v18+), npm/yarn.
- **Frontend Technologies:** React 18, TypeScript, Vite, React Router DOM.
- **APIs:** Google Maps Platform (Maps, Directions).
- **Data Processing:** Node.js (fs, node-fetch) for the geocoding and data normalization scripts.

3.2 Mapping between Requirements and Implemented Functions

The following table shows a mapping between the projected functionalities of the system and the corresponding implemented modules.

Table 1. Test Table

Functional requirements	Modules/Functions/Class that implemented this feature
Interactive Map Display	MapPage component, google.maps.Map instantiation
Smart Routing	RoutePlannerPage, calculateRoute(), Google DirectionsService
Station Filtering	MapPage, filter array methods based on greenFilter, radiusKm
Multi-language (i18n)	i18n.ts dictionary, LangContext, t() translation helper
Station Search	search state in MapPage and StationsPage comparing strings
Distance Calculation	kmBetween() utility using the Haversine formula

3.3 Implementation Details

The application starts at main.tsx, which mounts the React application wrapped in BrowserRouter and LangProvider. The main GUI interface is defined in App.tsx, which acts as the layout container with a premium Navbar and a dynamic <Routes> setup.

Main Logic - Smart Route Filtering: When a route is calculated, the system extracts the path coordinates. To optimize performance against 15,000 stations, it first creates a bounding box around the route to filter out distant stations instantly. Then, it iterates through the remaining candidates, calculating the exact distance to the route polyline using the kmBetween() utility.

Figure 1. Sample code for creating and connecting to a socket

```
// Sample code snippet for Route Optimization logic
const candidateStations = stations.filter(s =>
  s.lat >= minLat && s.lat <= maxLat && s.lng >= minLng && s.lng <= maxLng
);

const nearbyStations: Station[] = [];
for (const s of candidateStations) {
  let minDistance = Infinity;
  for (let i = 0; i < coords.length; i += Math.max(1, Math.floor(coords.length / 200))) {
    const d = kmBetween(s.lat, s.lng, coords[i].lat, coords[i].lng);
    if (d < minDistance) minDistance = d;
  }
  if (minDistance <= maxDistanceKm) {
    nearbyStations.push(s);
  }
}
```

3.4 Actual Database Schema

ChargingStations Collection (Document Structure):

- id (String): Unique identifier.
- name (String): Station name.
- lat (Number): Latitude.
- lng (Number): Longitude.
- brand (String): Operator brand.
- serviceType (String): e.g., "Halka Açık" or "Özel".
- maxPower (Number): Maximum kW output.
- isGreen (Boolean): True if powered by renewable energy.
- acPrice / dcPrice (Number): Pricing details.
- socketCount (Number): Number of available plugs.

4. Testing

4.1 Expected Test Scenarios

1. **Map Rendering:** Verify that 15,000 station markers load without crashing the browser.
2. **Filter Accuracy:** Verify that selecting "Green Energy" hides all standard stations.
3. **Route Generation:** Verify that entering "Istanbul" to "Ankara" draws a polyline and filters stations correctly.
4. **Localization:** Verify that clicking the language toggle translates the Navbar, Map Sidebar, and Route Planner instantly.

4.2 Unit Test

- **Distance Utility (kmBetween):** Tested with known coordinate pairs (e.g., Istanbul to Ankara) to verify accuracy within a 1% margin of error compared to standard Haversine calculators.
- **Geocoding Script (geocode-stations.mjs):** Verified that string addresses were successfully converted to latitude/longitude floats using the Nominatim API.

4.3 Functional Test

Functional testing was performed manually through the browser interface:

- **Routing:** Successfully generated a route between two cities. Handled edge cases where invalid locations triggered user-friendly alert messages.
- **UI Responsiveness:** Resized the browser window to simulate mobile devices; verified that the sidebar correctly transitions into a bottom sheet drawer.
- **i18n Translation:** Confirmed that dynamic numbers (e.g., "Filtered (1,200)") correctly format according to the active locale (tr-TR vs en-US).

4.4 Usability Test

We navigated the application as a prospective EV driver. The switch from Leaflet.js to Google Maps significantly improved the panning and zooming smoothness. The premium dark mode aesthetic reduced eye strain during nighttime use scenarios, and the instantaneous language toggle made the app accessible to tourists driving in Turkey.

5. Deployment of the System

The application is structured for easy deployment on static hosting platforms.

1. The source code is built using `npm run build`, outputting optimized static assets via Vite.
2. The `dist/` folder can be hosted on platforms like **Vercel**, **Netlify**, or **Firebase Hosting**.
3. *Deployment Architecture Diagram (Description)*: Client browsers request the SPA from the CDN. The SPA queries the Google Maps API directly. Data is currently bundled statically but is prepared to fetch from a Node.js/Firebase backend via standard REST calls in the future.

6. Limitation of the System

- **Static Data:** Currently, the application relies on a compiled JSON file. Real-time availability (occupied/free status) requires the backend MQTT IoT integration, which is planned but not fully implemented in this phase.
- **Performance with 15k Markers:** While highly optimized, rendering 15,000 native Google Map DOM markers simultaneously can cause slight DOM lag on very low-end devices. Implementing server-side clustering or Canvas-based markers is a future optimization.
- **Backend Features:** Payment gateways (Stripe) and user authentication (Firebase Auth) were designed in the SDD but deferred to the next development cycle to prioritize the core map and routing algorithms.

7. Conclusion and Future Work

The ECNA project successfully establishes a robust, highly functional, and aesthetically premium frontend platform for EV drivers in Turkey. We achieved our primary goals of interactive map discovery, smart route planning, and complex data filtering within a multi-language environment.

Future Work:

1. **Backend & Database:** Deploy the Firebase Firestore database and Node.js REST API to serve dynamic station data.
2. **IoT Integration:** Implement the MQTT broker to receive live availability statuses from charging stations.
3. **User Accounts & Payments:** Add Firebase Authentication for user profiles and integrate Stripe for reservation payments.
4. **Mobile Native App:** Wrap the React application using React Native/Expo or Capacitor for native iOS/Android App Store deployment.

8. Reference

1. International Energy Agency (IEA). (2023). Global EV Outlook 2023.
2. Azzopardi, B., & Gan, C. K. (2022). Smart Charging Infrastructure for Electric Vehicles. Renewable and Sustainable Energy Reviews.
3. Google Maps Platform Documentation. (2024). Maps JavaScript API & Directions API.
4. React Documentation. (2024). <https://react.dev/>
5. Vite Documentation. (2024). <https://vitejs.dev/>